INTRODUCTION TO CONTAINERIZATION AND DOCKER BASICS

WHAT IS MONOLITHIC AND MICROSERVICES APPLICATIONS

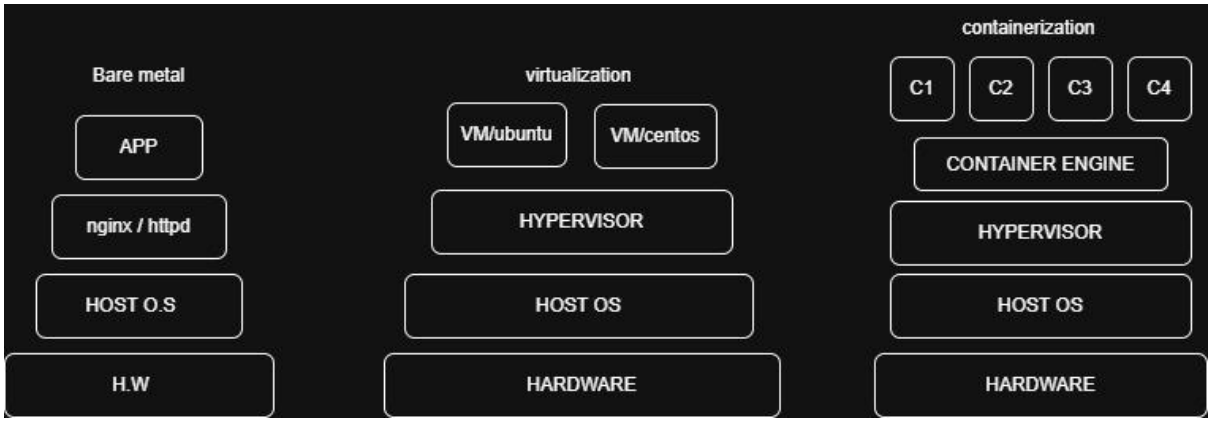
Monolithic

In the monolithic application all the functionalities of a project should exist in single codebase (means in a single file) then that application is known as monolithic application.

Microservices

it is a software architecture pattern where an application is built as a collection of small, independant services.

DIFFERENCE BETWEEN THE BARE-METAL / VIRTUALIZATION / CONTAINERIZATION



BARE METAL at time of monolithic application

1.In the beginning, there was no cloud.

- Companies used one physical server
- Installed OS → Installed application

2.Example:

Flipkart runs on one server

3.Problems:

- Only one app per server
- If app crashes → whole server down
- Cannot scale easily
- Resources wasted (CPU/RAM idle)

4.Conclusion:

Not suitable for modern apps

VIRTUALIZATION At time of microservices

- One server → many VMs
- Each VM has its own OS + app

1.Example:
Flipkart splits into:

- VM1 → Login
- VM2 → Payments
- VM3 → Products

2.Benefits:
1. Better resource usage
2. Isolation

3.Problems:
1. Each VM has full OS → Heavy (GBs)
2. Needs more RAM/CPU
3. Slow startup
4. If 40 microservices → 40 VMs

4.Key Issue:
Microservices are lightweight, but VMs are heavyweight

CONTAINERIZATION for the microservices

- Share same OS
- Only app + dependencies inside container

1.Example:
Flipkart microservices:

- Cart → Container
- Payment → Container
- Product → Container

2.Benefits:
1. Lightweight
2. Fast
3. Less RAM usage
4. Perfect for microservices

3.But problem:
1. Hard to create/manage
2. No standard tool

DOCKER

- Easy to create containers (Dockerfile)
- Easy to share (Docker Hub)
- Fast deployment

- Reusable images

Why Docker?

1. Easy to use container technology
2. Uses less RAM/CPU
3. Runs directly on Host OS
4. Images can be reused many times
5. Containers start in seconds
6. Perfect for microservices

WHAT IS CONTAINERISATION

Containerization is a technology where we use tools like Docker to create multiple lightweight containers that share the same operating system and run applications efficiently with limited resources, making it ideal for microservices architecture.

1. Bare Metal (Traditional Setup)

- Application runs directly on the server without any virtualization.
- Software like nginx/httpd runs on top of the Host OS.
- There is no isolation between apps, so conflicts can happen.
- Hardware → OS → App (simple but less flexible).

---

2. Virtualization

- A hypervisor creates multiple virtual machines (VMs).
- Each VM has its own OS (Ubuntu, CentOS, etc.).
- Strong isolation, but uses more memory and resources.
- Hardware → Host OS → Hypervisor → VMs → Apps.

---

3. Containerization

- Uses Docker engine to run containers instead of full VMs.
- Containers share the same OS but are isolated.
- Lightweight and faster than virtual machines.
- Hardware → Host OS → Docker → Containers → Apps.

Quick Difference

- Bare Metal → Simple but no isolation
- Virtualization → Full isolation but heavy
- Containerization → Lightweight + fast + efficient

DOCKER

docker is a free and open-source platform as a service model that uses the os-level virtualization to create, deploy and run the application in a single host os without creating the fully virtual machine.

HISTORY

1. it was first release in march 2013 and it is developed by solomon hykes and sebastian pahl.
2. it is a platform as a service model that uses os-level virtualization.
3. whereas vm uses hardware level virtualization.

DOCKER ARCHITECTURE

Docker Client → Docker Daemon (Engine) → Docker Registry
↓
Images ↔ Containers

Docker Client

1. Docker client sends commands to Docker engine using API.
2. Run commands like
    1. docker run
    2. docker build
    3. docker ps

Docker Daemon

1. It runs continuously in the background and listens for requests from the client.
2. It is responsible for creating, running, and managing containers.
3. It builds and stores Docker images used to launch containers.

Docker Engine

1. Docker Engine is the core platform that enables containerization.
2. It provides the environment required to build, run, and manage containers.

Docker Registry

1. it is a storage for the docker images

DIFFERENCE BETWEEN DOCKER CE AND DOCKER EE

DOCKER CE

- Free and open-source version of Docker
- Best for students, developers, and small projects
- Gets regular updates but no official support from Docker company
- Used for learning, testing, and development work
- Docker CE is free and best for practice and small applications

DOCKER EE

- Paid version provided by Docker Inc.
- Designed for large companies and production environments
- Comes with security features, stability, and official support
- Certified to run on platforms like Windows Server and Red Hat Enterprise Linux
- Docker EE is paid and used by companies for secure and reliable applications

---

HOW TO INSTALL DOCKER

1.create a instance

2. name --------> docker
3. machine ----> ubuntu
4. sg ------------> http/https/ssh
5. create a instance

2.access the instance & install docker

1. setup docker repository

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update
```

3. sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
4. sudo systemctl status docker

3.Run our 1st container

1. docker ps
2. docker run nginx:latest
   {This container run in the foreground – simply occupy the terminal}
3. docker ps -a
4. docker run -d nginx:latest
5. docker ps

Note:

1.docker ps --------> to check the running containers
2.docker ps -a -----> to check all the containers including the stopped containers
3.docker run -d ---> to run container in the background
